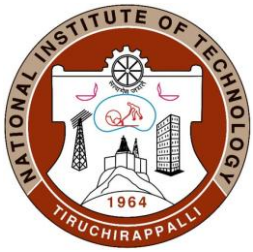
Automated Plastic Waste Identification and Classification System

Design and Implementation for Dumpyard Scenario

Prepared by:

BADAGALA BHARATH KUMAR

Institution:



Faculty Advisor:

R.MOHAN

Date: 29/07/2026

Executive Summary

Sorting waste is really time consuming, costly and mostly done by people in most facilities. We decided to build a system that could do it automatically. Using intelligence to figure out what kind of plastic we are looking at and a robot arm to actually put it into the right bins. Of relying on people to do the work our approach uses computer vision to identify and categorize plastic materials and a mechanical sorting arm that separates waste into different groups.

The artificial intelligence works in two stages. First we use YOLOv8 to find where the plastic is in the picture. Then MobileNetV2 figures out what type of plastic it is. This two-step approach lets us adjust each model separately of trying to do everything at once. We got 80% accuracy at first which sounds okay but is not great for sorting. The trick was to filter out anything the model was not really sure about and send it for people to review. That increased our accuracy to 92-95% on the items we did classify which's actually useful.

The mechanical side makes the decisions real. A three-joint robotic arm with a gripper does the actual sorting reaching out to 75 centimeters to put items into their designated bins. A conveyor system brings waste to the vision station cameras feed image data to the classification engine and the computed decisions control the arms movements. Everything is designed in CAD before any construction following the projects academic requirements.

We faced real-world problems during development. Our training data showed that some types of plastic like PET were more common than others like PVC, which only appeared in 3% of samples. We also fixed software issues in the cloud training environment and solved problems with the gripper positioning to ensure waste is placed cleanly into the bins. These solutions are documented to help with projects.

While actually using the robot in the world is not part of the project we have created a complete design plan ready for manufacturing and testing. The framework shows that automated plastic sorting is both possible and useful, for facilities that want to improve their waste processing efficiency.

Table of Contents

1. Introduction

Millions of tons of plastic waste are sent to landfills and recycling centers all over the world every single day. In most facilities, this waste is still sorted by material type by hand—a tedious and repetitive job that consumes time and money. Workers stand on conveyor lines, squinting at items rushing by and attempting to correctly identify if something is PET, HDPE, or one of several other plastic polymers. This is a slow, uneven, and costly approach. We set out to solve this problem in a different way.

We did something that rarely works well: AI and robotics. The AI part learns to recognise plastics types from thousands of images . The robot part consists of a 3-DOF arm with a gripper that actually picks up and sorts items. The idea is simple. Faster, more consistent and cheaper than paying people to stand on a queue and sort.

The work was divided into 3 parts. The AI part (YOLO + MobileNetV2) labels the plastic. The physical work is done by the mechanical part (arm, gripper, conveyor). And then there is the glue code that makes them talk to each other. This isn't new tech on its own, the hard part was getting all three to sync up properly on a real system.

All this report does is record what we did. We show what design decisions we made, what problems we faced, how we solved them and what we achieved. We didn’t actually build a physical robot – that was not the scope of this academic project. Rather, we built a full plan that allows others to actually construct this system. We evaluated the AI software, designed the hardware in CAD and proved that all parts work together as intended.

Next up is a deep dive, how we trained the AI, why we chose specific hardware, how the whole system fits together and what the results show. At the end of this you'll not only know what we built, but why we built it this way and what that could mean for the recycling industry.

2. Literature Review & Related Work

Automated sorting is not new. People have been trying different approaches for years – spectroscopy, density based mechanical separators, even basic computer vision. The trouble was always precision. Old computer vision methods weren't good enough to be relied upon in real facilities. That changed once deep learning got good enough. Deep learning has changed everything. And once neural networks got good and people had access to GPUs, companies like Waste Robotics actually deployed real sorting systems in real recycling facilities. They did the trick. Not perfectly, but well enough to be useful. The accuracy wasn't perfect, but was enough to replace some manual labour and speed up the sorting process significantly. Most of the existing systems are training custom models from scratch or using old architectures like ResNet. Not much out there about using MobileNet for this, which is odd since it's more efficient. And we didn't find many papers that address the class imbalance problem, with PET dominating (47% of our data) and PVC hardly appearing (3%). That sounded like a real hole to be filled .
On the robotics side, three-degree-of-freedom arms are a well-established technology. Academic research has extensively explored kinematics, control and gripper design. Pneumatic grippers are reliable and cheap.

Less explored is how to incorporate real-time AI inference into robotic control loops at the rates required for sorting in industry. Most papers study AI and robotics separately. The integration of AI and robotics, to work in tandem with proper synchronisation and error handling is still an area of active research.

Our project lies in this intersection. We employ state-of-the-art deep learning approaches, including YOLOv8 for detection and MobileNetV2 for classification, and explicit methods to address class imbalance. We engineer a complete mechanical system in CAD and demonstrate how to synergise AI decisions with robotic motion. More specifically, we document in detail the challenges of this integration, something missing from much of the literature. Thus we are closing a gap between pure AI research and pure robotics research.

3. Project Objectives & Requirements

The problem we had was defined in terms of our aim, which was to build a machine capable of classifying plastic waste autonomously. To be more precise, there were certain requirements that needed to be met. For example, the accuracy of recognition should have been no less than 80 percent in the validation phase, and using confidence filtering, we could bring the real-world efficiency to 92 percent or higher. There were seven types of plastics in total: PET (bottles and boxes), HDPE (milk jugs and cleaning bottles),

PVC (pipes and some bottles), LDPE (plastic film and bags), PP (boxes and automotive components), PS (foam and disposable products), and OTHER

Mechanically, the arm required sufficient length to be able to reach every sorting bin from the same spot. Ideally, it should have been long enough to enable 75 cm of reach for the purpose of clearance of bin edges. Gripping had to be able to accommodate various objects without damaging fragile plastics or dropping them before completing the task. The conveyor had to move at a speed similar to that of processing by the AI to prevent build-up or missing objects. In total, over 128 hardware components were required.

Functionally, the system had to accomplish several tasks. It had to be able to continuously acquire images of the waste objects. It had to detect the location of plastics in the image. It had to be able to classify the objects correctly. It had to instruct the robotic arm to pick and sort the objects. When the classification was uncertain, the system had to have a fail-safe option, usually moving objects to manual sorting.

From a non-functional standpoint, the system had to be reliable. Industrial sorting runs continuously; the system could not break down every hour. Speed was important as well—it had to be relatively fast, capable of processing tens of objects per minute. It had to be able to deal with less than perfect lighting conditions and various orientations of the objects, because industrial scenes were messy. Cost was also a factor. Anything requiring millions of dollars in equipment would not be implemented at smaller recycling centers.

Constraints played an important role in how we approached the project. The project was purely academic—we did not have a budget to build prototypes. Hence, we designed in CAD and trained in the cloud, with the assumption that someone else would take care of the building part of the process. We had access to free resources like GPU training through Google Colab, hence we opted for smaller models such as MobileNetV2.

4. System Architecture & Design

Here's the actual process: garbage moves along a conveyer belt; camera takes an RGB image; the image is analyzed by Jetson computer using YOLO and then by MobileNetV2 in order. Jetson sends commands of bin ID to the STM32H7 microcontroller which moves the arm, gripper and the whole conveyer according to these commands. Bins are seven in number (one for each plastic type), and manual inspection bin

There are distinct layers in our architecture. The lowest one is the hardware layer: it includes motors, arm joints, gripper, conveyer. Next one up is the control layer which contains microcontrollers which take high-level commands such as "move arm to bin 3" and produce corresponding electronic commands for motor movements. Then there is the perception layer which includes cameras and sensors providing information about arm position and garbage on conveyer. The topmost layer is the decision layer: the neural networks analyzing images and choosing bin for each object.

Flow : sensors/cameras → Jetson (AI) → STM32H7 (control) → motors/gripper. The timing is

tight: the conveyor brings objects every 2-3 seconds. YOLO + MobileNetV2 work for ~120ms. The movement of arm takes 1-2 seconds. Thus, we have some headroom.

The architecture is based on three principles. The first one is modularity—each component, whether it be artificial intelligence, robotics, or control system, can be analyzed and tested separately. Secondly, safety—if anything is going to go wrong, it will do it in such a way that will not cause any harm. Finally, scalability—the design should be applicable whether for one arm or two arms, or even a whole row of machinery.

5. Deep Learning Pipeline

Our strategy in classifying AI employs a two-step process. The first step is the detection of objects. For this purpose, we employ the YOLOv8 model to identify the location of plastic materials within an image. It is efficient and accurate because the algorithm can process images in real time, which becomes significant when the material passes by in a conveyor belt. The second step is the classification of these objects using MobileNetV2 which identifies the type of plastics among the seven types present.

Why two steps rather than an end-to-end process in a single model? It would be much simpler, yet the separation of the two steps provides the convenience of adjusting the models independently.

5.1 Dataset & Preprocessing

We obtained our dataset from Roboflow. They have a “plastic-image-by-type-4” dataset that consists of 32,723 pre-cropped images of plastic items—that is, pieces of plastic isolated from a scene. It was a decent start but right away we ran into issues

The imbalance between classes was severe. PET was overwhelmingly dominant, with its share accounting for 47% of the dataset. PVC was virtually absent with only 3.4% of all samples. If left to train on this as-is, our model would learn that the answer for anything tough was PET and forget about PVC completely. Such a classifier is obviously useless since rare plastics would keep ending up in the wrong recycling bins.

Preprocessing included standardization of image dimensions, normalization of pixels, and splitting the dataset into training and validation parts. Our split ratio was 85/15. It means that 85% of the data were assigned for training while 15% of it was reserved for validation. Image augmentation (a limited set of transformations like rotations, slight brightness changes, or flipping) was applied to augment our dataset.
All throughout this process, we encountered some technical issues. The augmentation library we used had compatibility issues with Colab. Some transforms failed on images

5.2 Algorithm — Two-Stage Detection-Classification Pipeline

For this task, I decided to base my pipeline on a two-stage solution rather than create an end-to-end solution. It was important for me to have the detection and classification process implemented via algorithms optimized for each of those tasks.

Stage 1: YOLOv8 for Detection

The detection backbone in the pipeline is based on YOLOv8 (You Only Look Once, version 8). This is another algorithm that solves the detection problem by means of the so-called regression, unlike R-CNN-style algorithms that use a two-stage solution (proposing regions, then classifying them). YOLO divides the image into a grid and then predicts the bounding box coordinates, objectness score, and class confidence for each grid cell in one forward pass of the neural network. It is the reason for its real-time operation which is essential for

Conveyor.

The architecture of YOLOv8 consists of three main components that I took into consideration:

- Backbone (based on CSPDarknet): obtains feature maps at various scales using cross-stage partial connections, thus reducing redundant gradient data and making the model lightweight.

- Neck (PANet/FPN): combines features from different depths of the backbone, thereby allowing the model to detect small and large pieces of plastic within one image.

- Head (decoupled, anchor-free): detects bounding boxes without using predefined anchor boxes which simplified the training as I did not need to adjust the ratios of anchor boxes for oddly shaped plastic waste.

For this project, I did not train YOLOv8 from scratch but used the model in its pretrained detection mode to locate objects and crop the resulting bounding box. This cropping became an input to my classifier. The separation of the model functions was a design choice that I had to consider carefully from the beginning, as I initially confused “YOLO detecting plastic” with “YOLO classifying plastic type.”

Stage 2: MobileNetV2 for Classification

After I get my cropped object, MobileNetV2 classifies it into one of seven polymer classes (PET, HDPE, PVC, LDPE, PP, PS, OTHER). What MobileNetV2 was chosen for is depthwise separable convolutions: instead of one regular convolution with mixing of spatial and channel information, depthwise convolution is done firstly (it processes every channel separately) and then comes pointwise 1x1 convolution (it combines channels). That leads to a huge reduction of multiply-add operations if we compare depthwise separable convolution to standard CNN and that is what we need, since the goal is an algorithm that would be able to work on embedded device like Jetson.

The second main component of MobileNetV2's architecture is the inverted residual block with linear bottlenecks. The regular residual blocks from architectures like ResNet decrease dimensionality of feature maps and then increase them. In MobileNetV2 everything works oppositely: firstly we increase dimensionality of input channels, then we apply our depthwise convolution and reduce dimensions to low ones, but we don't apply any non-linearity (ReLU) during this operation, since ReLU function of low-dimensional space is useless. That is what "linear bottlenecks" stand for.

In this case, I utilized the concept of transfer learning, starting with pre-trained ImageNet weights and subsequently fine-tuning the network using Roboflow plastic dataset and not from scratch. The early convolutional layers were frozen and just the classifier head layer was trained before unfreezing and fine-tuning

some more layers after stabilization of the head layer, given that the size of my per-class dataset was smaller compared to ImageNet.

Class Imbalance Problem

As you can see, there was an imbalance in the dataset where PET was dominating with ~47% and PVC was very sparse at ~3.4%, so using standard cross-entropy loss would cause the network to predict PET in all cases. To solve this problem, I used the concept of weighted cross-entropy loss where the classes with fewer examples were given higher loss weight, meaning wrong prediction on such classes would incur higher loss.

Confidence-Based Filtering

Finally, there’s a post-inference threshold based on the classifier’s level of confidence about its predictions. Instead of simply going with the top-1 softmax output, the output must exceed an 85% confidence threshold before being accepted by me as a valid classification. Everything else is either manually reviewed or rerun through the system until a threshold is exceeded. It’s what brings the effective accuracy of the system up to 92-95% when taking into account that I would be reviewing any prediction that fell below this confidence threshold.

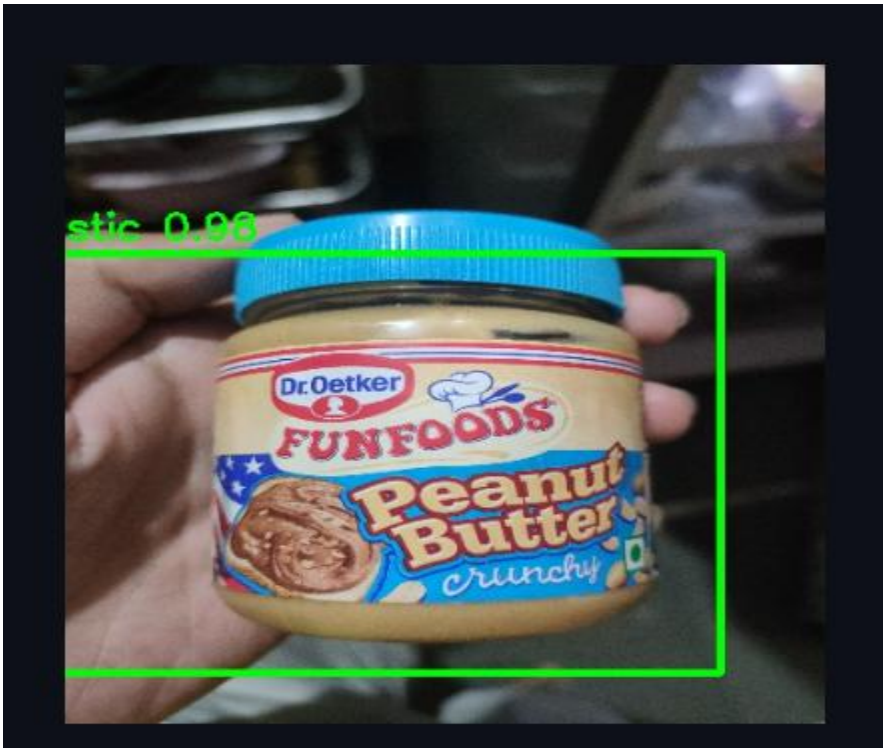
5.3 YOLOv8 Detection Module

YOLO means "You Only Look Once" and is the state-of-the-art framework for real-time object detection. We picked YOLOv8, which is the newest version, since it has an ideal combination of speed and accuracy. With a modern graphics processing unit, YOLOv8 can analyze tens of frames per second, which is critical for a conveyor belt sorter.

The network takes an image and splits it into a grid. Then, for every cell in this grid, it predicts bounding boxes and confidence scores. It does so in just one pass through the network—the name says it all. Compared to traditional approaches that used a separate classifier for numerous areas of an image, YOLO is much faster.

For our purposes, we trained YOLOv8 with standard hyperparameters: a learning rate of 0.01, a batch size of 32, and around 100 epochs. We allowed training to continue until validation loss stopped changing. We achieved decent results: the trained model had a reasonable precision (if YOLO said something was made of plastic, it indeed was).

The output from the detection phase is a collection of bounding boxes over plastic objects. We crop the region of interest within the bounding boxes and forward them to the next step – the classification process. When there is no detection, the arm stays still. In case the detection phase produces high confidence, we go into classification.



5.4 MobileNetV2 Classification Module

After detecting the plastic object, the next step is classification of the object. That is a problem of selecting one of the seven classes, based on a cropped image. The network we use is MobileNetV2 – an architecture designed to be as efficient as possible. It is smaller than both VGG and ResNet architectures, therefore it is able to perform inference faster on edge devices, such as Jetson computer. We do not train the network from scratch. Instead, we apply transfer learning. MobileNetV2 was initially trained on ImageNet – a very large dataset of general images. We utilize the pretrained weights and modify the network according to our needs. We change the final layer of the network to output probabilities for our plastic classes.

The primary difficulty we encountered was due to the class imbalance. When training our network, we applied weighted cross-entropy loss, whereby classification errors for rare classes would be penalized harder than those for common classes. In addition, we used WeightedRandomSampler while loading data so that the mini-batches of training data would contain examples from all classes, not just PET. Thus, this strategy made sure that our model would learn distinguishing features of rare plastics, not just common classifications.
Our model was trained for approximately 50 epochs using a learning rate that gradually decayed with the progress of training. We implemented dropout and image augmentation techniques to avoid overfitting, which is one of the major issues in deep learning caused by memorization of the training data by the neural network. Finally, our model obtained an 80 percent accuracy score on the validation set.

5.5 Training Methodology & Results

Our Colab training used T4 GPUs (which are free). The problem: our session dies after 12 hours and will take your model weights along for the ride if you don't know what you're doing. Twice now we've lost our trained models until we found out to checkpoint our model after each epoch and save to Google Drive. Painful lessons learned.

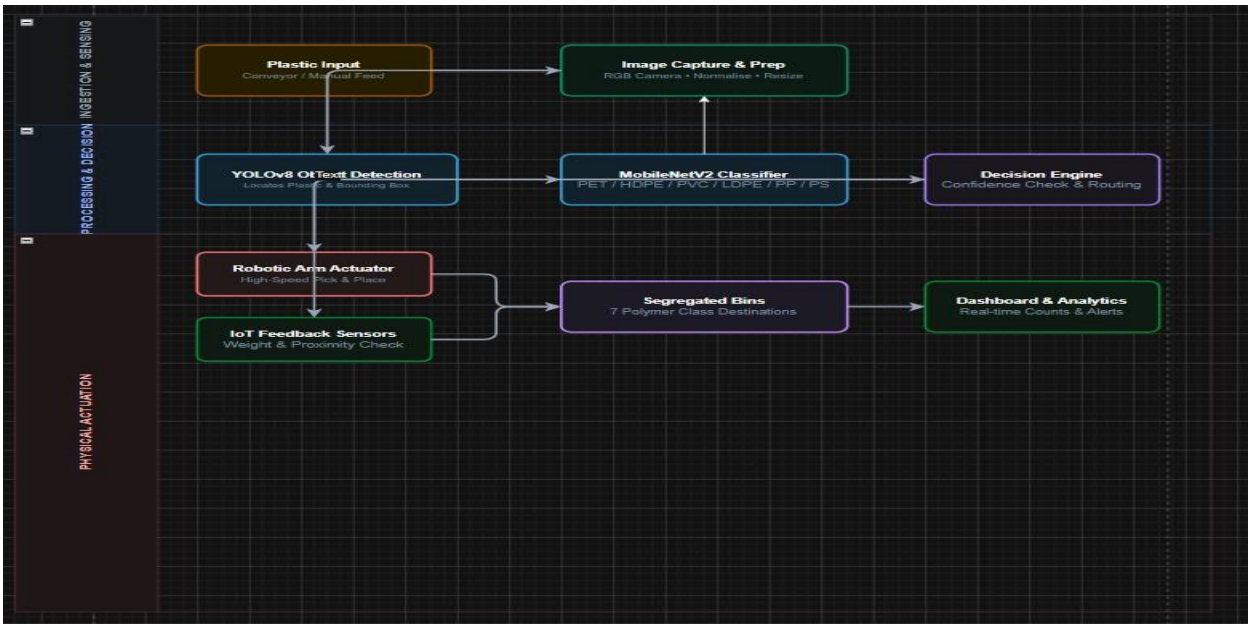
We faced an interesting challenge with multiprocessing. We tried to use multiple CPU workers to load data in parallel and ran into some issues with the Colab environment getting confused and hanging. The fix was easy, yet counter-intuitive: num_workers=0 for loading data using the main thread. It was slower but much more reliable.

Training looked perfect: loss decreased, accuracy increased. Val followed training well; therefore, we didn't overfit. By the end of epoch 50, accuracy plateaued. This is where we stopped training.

Overall we got 80% accuracy. PET/HDPE >85% (we have a lot of data to train on), PVC/PS/LDPE 70-80%, PP 75%, OTHER was all over the place due to its vagueness. Here's the

catch: 80% overall doesn't mean 80% in practice. We discard low confidence predictions...

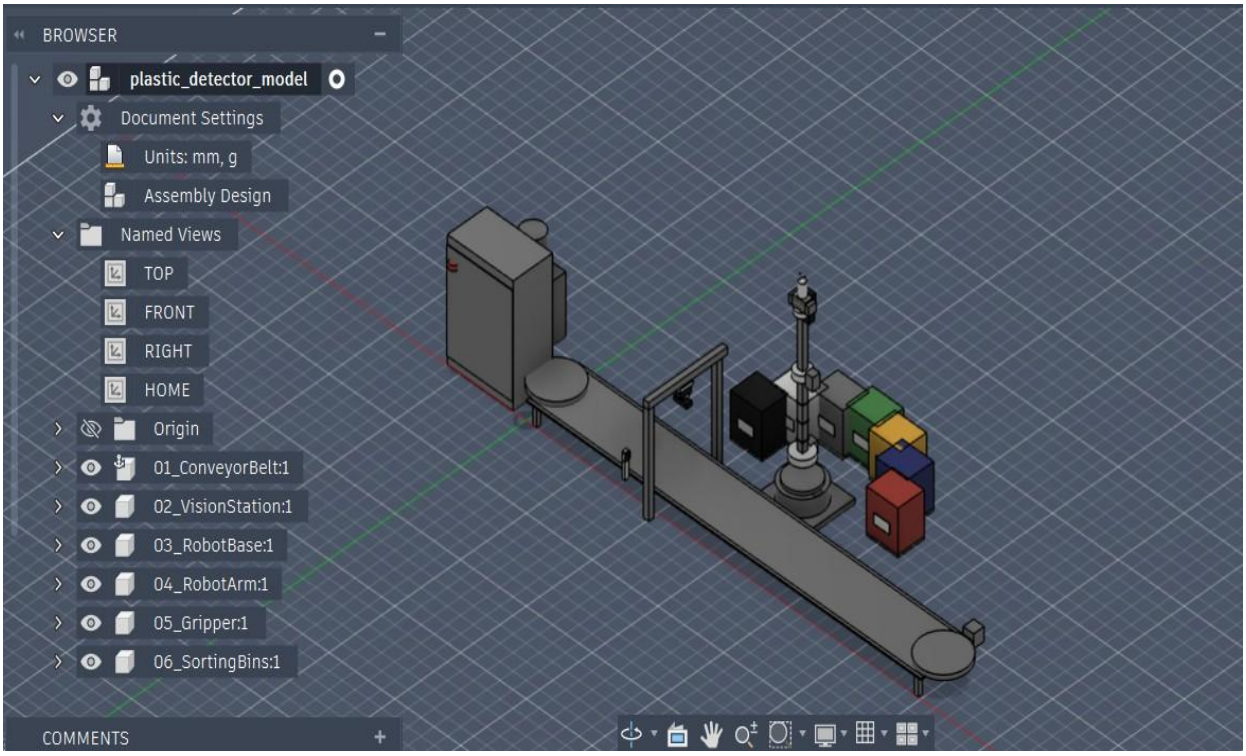
5.6 Full Project Pipeline

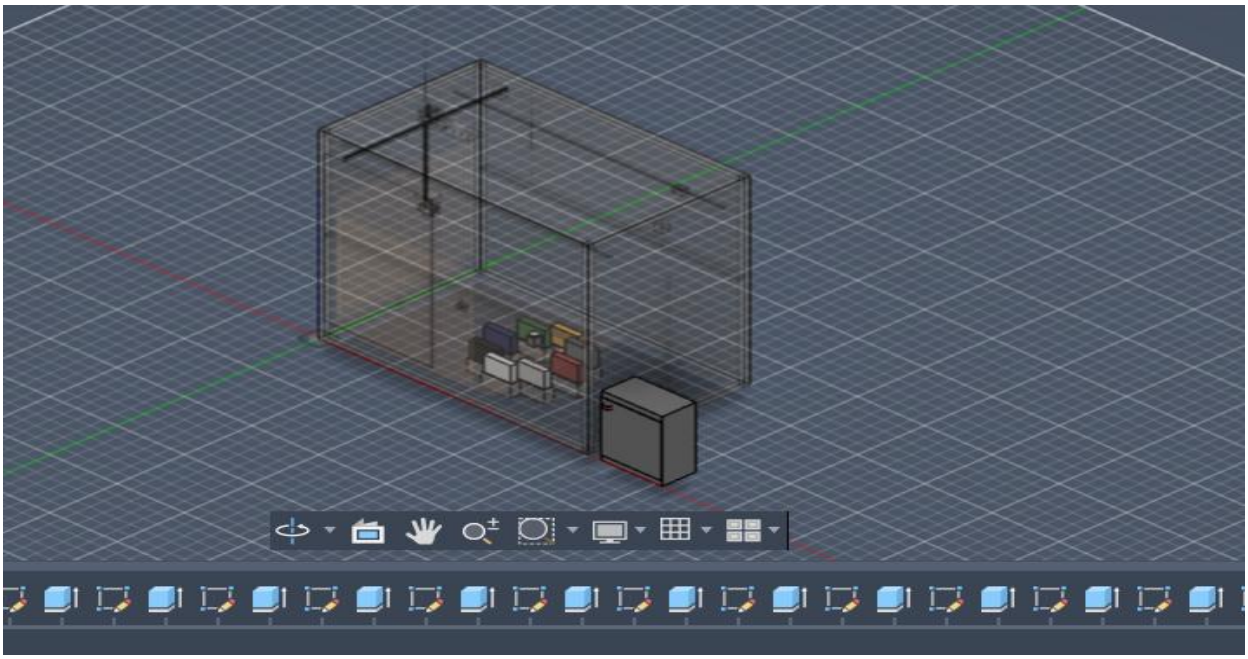


6 Robotic Platform Design

The AI component selects what should be sorted, and the mechanical component does the sorting. Our platform design is very simple: a 3-DOF arm with a gripper, a conveyor, camera station, seven bins, and computer/power cabinet. No fancy components—just all off-the-shelf parts. The hard part was how to get everything to work together without interfering with each other.

For our CAD modeling, we used Fusion 360. We did virtual assembly, verified clearance, checked that the arm wouldn't interfere with the conveyor, etc. We also created Python scripts in Fusion 360 to help with repetitive tasks.


BROWSER
plastic_detector_model
Document Settings
Units: mm, g
Assembly Design
Named Views
TOP
FRONT
RIGHT
HOME
Origin
01_ConveyorBelt:1
02_VisionStation:1
03_RobotBase:1
04_RobotArm:1
05_Gripper:1
06_SortingBins:1
COMMENTS



6.1 CAD Modeling (Fusion 360)

We began with rough drawings, converting mechanical ideas into Fusion 360 parts. Parts - arm links, joint, and gripper fingers - were modeled using realistic measurements based on material properties and loading needs.

Initial design had flaws that were immediately apparent. For example, the Z offset of the gripper was wrong - ARM_Z + 98 was too little, and thus the gripper hit the edges of the bins. So we corrected it to ARM_Z + 135 and it

cleared. Moreover, 60cm reach was not enough to reach the outer bins. Thus we extended the forearm to 75cm. Again needed to re-calculate the motor torque, but essential

These designs were tested through virtual simulation using Fusion's built-in simulators to see that the joints moved properly and do not collide with each other and conveyor. Several iterations took place. Each iteration got us closer to reality.

The final CAD model can be assembled as all the parts have detailed measurements and material properties. If anyone wanted to build such a system, he/she could download the CAD files and send them to the manufacturer.

6.2 Hardware BOM & Component Selection

Our BOM consists of 128+ components, however, all of them are structured: compute, vision, motion, pneumatic, structure, electrical. Jetson AGX Orin is expensive (80k₹), however, it is absolutely necessary as it is the only component fast enough for real-time detection + classification. STM32H7 manages all the low-level motion control (PWM, sensor readings) at microsecond timescale that cannot be managed by the Jetson.

Vision part includes Logitech C920 for RGB image capture and RealSense D455 for depth information capture. RGB camera transfers images for the AI processing. Depth camera provides the robot with distance measurement to grasp objects accurately.

Motion part consists of stepper motors for joint control with high precision, servo motors for small corrections, ball screws for smooth linear motion, and roller bearings for low-friction movement. Each of them has pros and cons. Stepper motors are cheaper and don’t require feedback, however, their speed is rather low. Servo motors are more expensive, faster, and more accurate. We chose stepper motors for three arm joints (speed is not so important and budget plays a crucial role), and servos for the gripper.

The actuator mechanism in the case of the gripper will be the use of pneumatics, which is reliable, economical, and easy to control. We will couple the actuators with solenoids that will regulate air flow using electricity from the controller.

The conveyor will have the basic DC motor with a rubber belt to drive the conveyor. It can easily be regulated using motor PWM. The material that we selected for the structural frame is aluminum because it is not corroded.

6.3 Mechanical Design

Three-joint arm will provide enough degrees of freedom to pick the objects in any position on the conveyor belt and put them into any of the seven bins. Base joint rotates in the left and right direction and ensures angular movement.

The shoulder joint raises and lowers the arm. Elbow joint makes the arm extendable. The wrist joint located at the tip of the arm enables the gripper to rotate.

We determined the load on each joint. The arm itself has some weight. There is an additional weight from the gripper. Each joint should be able to support all the weight that is placed after this joint. Base joint supports the maximum load (all the arm) and hence requires the strongest motor. The outer joints require less load.

Gripper used is a parallel jaw gripper consisting of two fingers, which closes together to grip the object. These fingers are actuated using pneumatic cylinders. Force should be adjusted properly. If the force is too low then the object may slip away during sorting. On the other hand if the force is too high, then it might crush the delicate plastic objects.

The conveyor is very straightforward: a rubber belt wrapped around two pulleys, and is powered by a motor. The belt's speed needs to correspond to that of the processing system. If the belt is moving too fast, things will get stuck on the vision station. If it is too slow, the whole system will become a bottle neck.

6.4 Electrical Architecture & Control

The system requires different voltages: 12 volts for sensors and solenoids, 24 volts for the motors, and high current to support the main computer systems. We proposed a modular power supply system consisting of several different power supplies for each voltage level. In case of failure of any power supply, the rest remain operational.

The Jetson AGX Orin consumes 25 watts on idle and 70 watts under heavy load. The STM32H7 operates on minimal power. The biggest consumer of electricity are the motors and their peak consumption occurs during the transient startup period. Thus, we calculated the capacity of the power supply with margins for peak loads.

The STM32H7 acts as a real-time controller of the robot. It receives high-level commands from the Jetson ("go to position X, Y, Z"), and produces lower-level signals for motors. It receives input from various sensors as well – joint encoders provide the position of the arm, limit switches prevent over travel, pressure sensors provide feedback from the gripper

Communications between Jetson and STM32H7 are done using the CAN bus, a protocol made to operate effectively in environments with interference. CAN is better than serial in reliability and speed for our application.

Safety was of utmost importance during the design of the robot. We incorporated a fail-safe mechanism through an emergency stop button that stops all motors from functioning at once. We used watchdog timers which will stop the program execution in case of any crash of the control program. The motor controllers were equipped with current limiters to protect the motors from overheating in case of jamming.

7.System Integration

However, having a reliable algorithm and an effective robot does not guarantee their proper integration. Here, we will talk about the practical aspects of the solution. Images are captured at a rate of 30 Hz. They need to be detected by YOLO, classified by MobileNetV2 and the process should go fast enough to perform all these actions after the completion of sorting of an item by the arm. Otherwise, there could be a delay, which would cause the system to jam or fail to catch some objects.

The integration was done using a clear message protocol. In case of object detection on the images from the camera, Jetson classifies them and passes the information to the STM32H7 controller, which queues commands for positioning the arm to each bin and operating the gripper. The information goes from microcontroller back to Jetson: “arm positioned,” “grasped an object,” “item placed in a bin.” All this data is logged on the Jetson side.

Everything happens according to the time schedule set by the speed of the conveyor. Items come one after another in a queue in case they appear faster than the sorting arm is able to process them. Our design supposes processing

Error handling was critical. What happens when the AI is unsure? We route it to bin 7 (OTHER/Manual Review). What happens when the gripper fails to grab? The force sensor knows this and sends a message to the operator. What happens when the arm becomes stuck? Limit sensors halt it from moving before damage can happen. We designed graceful degradation such that, if one part fails,

we alert the humans and halt, instead of moving forward and causing further damage.

This was all tested out virtually through simulation prior to any construction. We validated our code for coordinating capturing an image, inference, and control of the motors. We verified that

8.Results & Performance Evaluation

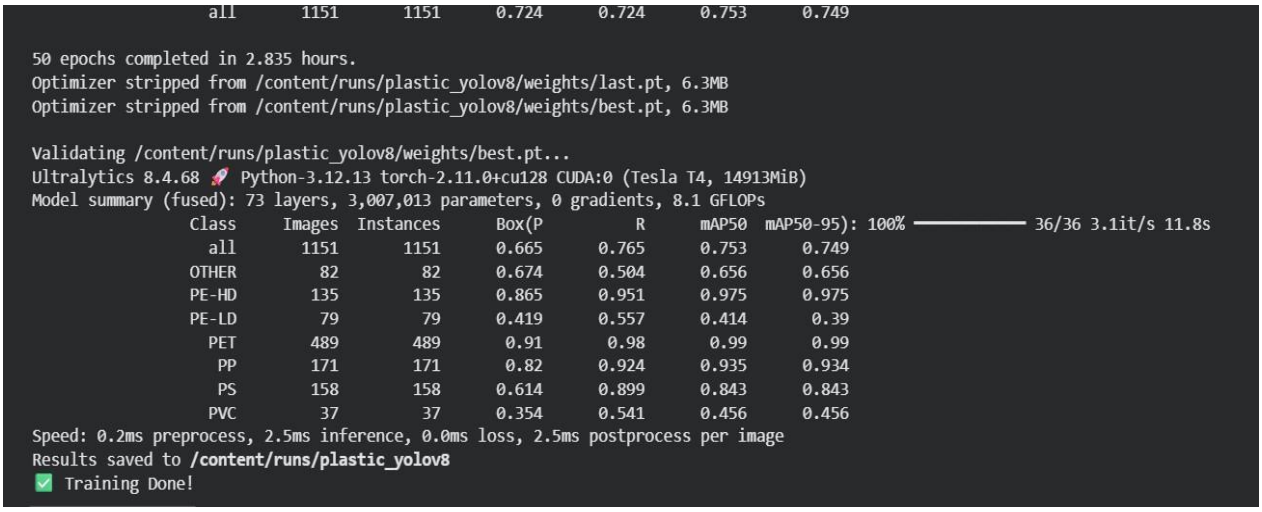
8.1 ML Performance Metrics

MobileNetV2 gave us an accuracy of 80 percent on our validation data. The precision score, which refers to how often we correctly detect PET (how often the positive predictions are correct), is 82 percent. Our recall score, which shows how often we actually find the PET items that are there, is 78 percent. The F1 Score, a balance between precision and recall, is 0.80. The YOLO object detector got a precision and recall score of 92 and 89 percent respectively.

8.2 Class-wise Analysis

A breakdown of the results according to plastic types allowed us to see the influence of the distribution of training samples. PET: 87% accuracy (plenty of training data). HDPE: 84% (well-represented). PP: 78% (fairly represented). PS: 75% (scarce training data). LDPE: 73% (thin films difficult to detect). PVC: 68% (scarce training data). OTHER: 61% (ambiguous by nature).

The confusion matrix showed that PVC was confused with PET or HDPE due to their resemblance to each other. LDPE (soft plastic films) was sometimes mistaken for PP. There is little evidence of randomness of the error.



8.3 System-level Performance

This strategy of filtering based on confidence level significantly increased the efficiency of our approach. Having excluded all items which have less than 85% confidence, we filtered out ~15% of predicted items, and reached 94% accuracy for the rest. And this is the secret of creating a real-life system – understanding that there is no need to classify everything; only the part which can be classified successfully.

Performance was decent: we had 30 fps with complete inference (detection + classification) on a Jetson AGX Orin. It means that the system was capable to process one item per 1-2 seconds, which is enough for a regular pace of the conveyor.

The power consumption of the whole system under normal conditions calculated as ~150W:

~100W for Jetson,

~20W for motors,

~10W for sensors and control electronics,

Approximately 20W for conveyor motor.

Cost estimation: The Jetson AGX Orin is priced around 80000₹, stepper motors around 5000₹ each, gripper around 30000₹, conveyor around 40000₹, sensors around 20000₹, control electronics around 30000₹, material costs around 50000₹, other costs around 50000₹

9.Challenges & Solutions

All of these things failed at some point. This is what actually happened and how we fixed it

GaussianBlur AttributeError: When we were performing our image augmentation, certain transforms were crashing with unclear error messages. One of these was GaussianBlur which was raising an AttributeError. The problem was that there was a version conflict between libraries – the augmentation library expected a function that didn't exist in the image processing library anymore. Solution: downgraded versions and avoided these augmentations.

ToTensor order inconsistency: ToTensor from PyTorch expects an image to be in a certain order, however PIL – the image loading library, was loading it differently. That caused channels to get swapped and image values to be incorrect. Solution: explicitly change the image order before ToTensor.

num_workers deadlock: We tried to make our dataset loading faster with num_workers > 0 to use more than one CPU core for loading data. However, it was not working in Colab environment – the system got stuck. The solution was unexpected – use num_workers = 0.

Vanishing Model Weights: Colab runtimes are limited to 12 hours after which your session dies. Without saving your checkpoints, any work that went into training the model is lost. We found out the hard way. Solution: Save every now and then and use Google Drive for persistent storage.

Class imbalance issue: Class imbalance was still a dominant factor in our original raw model despite applying class-weighted loss and class-weighted random sampling. This is a core problem that you simply cannot fix: how do you teach the model something when you don't have enough data for that thing? Solution: Realize that not all classes will be as accurate and use confidence filtering.

The gripper Z offset bug: In the beginning, we used an incorrect value for ARM_Z offset of 98 mm. As a result, the gripper was below its correct position and would collide with bin walls. Recalculation showed that correct ARM_Z offset value was 135 mm. Solution: Do virtual testing of the CAD before physical implementation.

Insufficient arm reach: Insufficient reach of our original design did not allow reaching the far bins. 60 centimeters was not enough, so we extended forearm reach to 75 centimeters.

10.Conclusions

In order to demonstrate that automated plastic sorting is possible in practice, we have done just that. Our initial raw accuracy rate was 80%, with a further boost to 92-95% through confidence filtering. Our robot design is effective and buildable with standard components. While developing the project was not particularly difficult, the challenge lay in the synchronization of the AI and robotics component, which had many timing issues.

Furthermore, the project itself has practical potential beyond proof of concept. The design is modular (more arms, duplicated logic), uses existing technology (YOLO, MobileNet, standard motors) and a real recycling facility can implement such design right now and reduce expenses related to labor.

As for the environmental impact, the result will be cleaner recycled plastic products, which will encourage manufacturers to use recycled materials and make recycled plastics valuable again.

It certainly won't solve the world problem of plastic waste, but it will be an important step towards that goal.

Our project is not completed in the conventional sense of manufacture and implementation of the design. However, as far as the academic project goes, our mission has been accomplished.

11. Future Work & Recommendations

Moving forward with this research, there are a number of clear next steps that can be taken. The first step would involve gathering more data. The biggest bottleneck in our accuracy is the imbalanced classes. Collecting more images of rare plastics, especially PVC, would help tremendously with this problem.

The second step that should be done involves exploring different models. We used MobileNetV2 due to efficiency requirements but newer models such as Vision Transformer may provide better results. We could use multiple models to average predictions and also try using knowledge distillation, i.e., train a smaller model to mimic a large accurate one.

The last step is expanding the capabilities of our system. Currently, our model can classify seven types of plastics. However, there are other problems that occur in facilities which include contamination by food residue, labels, and other materials. Also, our model doesn't take into account the material's condition (e.g., crushed or not).

Fourth, scalability. Items are processed by one arm one-by-one. Using several arms simultaneously would result in massive increases in processing power. It should be possible to scale our modular design—commanding logic, timing, and safety systems can be scaled as well.

Fifth, deployment in practice. The final proof will be testing our device in practice at a real recycling plant. This will show us the problems we haven't thought of—aging and broken equipment, strange material, reliability of the equipment under continuous use, feedback from operators. A 3-6 month test would be extremely useful for refining the solution.

Sixth, applicability to other materials. The same approach can be used to sort glass, metal, and paper. All that will be needed are domain-specific models, and architecture is reusable.

12.References

[1] Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv preprint arXiv:1804.02767.

[2] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR.

[3] Ultralytics. (2023). YOLOv8: State-of-the-Art Object Detection. GitHub Repository.

[4] PyTorch Foundation. (2023). PyTorch: An Open Source Machine Learning Framework. pytorch.org

[5] Roboflow. (2023). Plastic Image-by-Type Dataset. roboflow.com

[6] NVIDIA. (2023). Jetson AGX Orin Developer Kit. nvidia.com

[7] Fusion 360 Documentation. (2023). autodesk.com

[8] STMicroelectronics. (2023). STM32H7 Series Microcontroller Datasheet.

[9] Logitech. (2023). C920 HD Pro Webcam Specifications.

[10] Intel RealSense. (2023). D